

Traveling Salesman Problem (TSP) Details

The Traveling Salesman Problem (TSP) Definition

Given a set of points and a starting location, find the shortest (closed) route that passes through all these points.

Simple Example

For the following discussion, suppose we want to find the shortest route in travelling between the 4 points P_1, P_2, P_3, P_4 . In addition, suppose we must start and end at P_1 .

In our code, each point will be a row of a matrix. Let's call this matrix $\mathcal{P}$.

The **route order** is a vector containing the order in which to travel. Since we must start at P_1 , every route order must start with a 1.

The index (ID) of each point is determined by its order in $\mathcal{P}$. So the initial route order, P_1, P_2, P_3, P_4 , would be $[1, 2, 3, 4]$.

In total, there are 6 possible ways arrange 2, 3, 4 after 1, as shown below:

- Route order 1: $[1, 2, 3, 4]$ corresponding to route $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$
- Route order 2: $[1, 2, 4, 3]$ corresponding to route $P_1 \rightarrow P_2 \rightarrow P_4 \rightarrow P_3 \rightarrow P_1$
- Route order 3: $[1, 3, 2, 4]$ corresponding to route $P_1 \rightarrow P_3 \rightarrow P_2 \rightarrow P_4 \rightarrow P_1$
- Route order 4: $[1, 3, 4, 2]$ corresponding to route $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_2 \rightarrow P_1$
- Route order 5: $[1, 4, 2, 3]$ corresponding to route $P_1 \rightarrow P_4 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$
- Route order 6: $[1, 4, 3, 2]$ corresponding to route $P_1 \rightarrow P_4 \rightarrow P_3 \rightarrow P_2 \rightarrow P_1$

Recall, the goal of the TSP is to find which of these routes produces the smallest total closed distance.

Next, suppose we found the total distance for each of these routes to be,

- distance(route 1) = 1150 miles.
- distance(route 2) = 1253 miles.
- distance(route 3) = 1333 miles.
- distance(route 4) = 1001 miles.
- distance(route 5) = 1561 miles.
- distance(route 6) = 1005 miles.

Then the solution to this TSP would be route 4, $[1, 3, 4, 2]$, since it yields the smallest total distance.

Brute Force (BF) Approach

The brute force (BF) approach to the TSP works exactly as described in the previous example. That is, it computes the distance between all the possible routes and returns the route that produces the shortest distance.

PROS: Easy to code, and guarantees the exact solution.

CONS: Only feasible for a small number of points (less than 15).

The CON for this method is so restrictive that it is never used. To see why, consider the number of possible routes.

For our simple example of 4 points, we can count the number of possible route in the following way:

- In the 1st spot, place a 1 (this is free). Number of choices = 1.
- In the 2nd spot, place either 2, 3, or 4. Number of choices = 3. Say it was 4.
- In the 3rd spot, place either 2 or 3. Number of choices = 2. Say it was 2.
- In the 4th spot, place the 3. Number of choices = 1.

Multiplying all the number of choices gives 6. No bad, but for say 15 points, we count like so:

- In the 1st spot, place a 1. Number of choices = 1.
- In the 2nd spot, place 2, 3, ... 15. Number of choices = 14.
- In the 3rd spot, place a value of 2, 3, ... 15, not choosen previously. Number of choices = 13.
- In the 4th spot, place a value of 2, 3, ... 15, not choosen previously. Number of choices = 12.
- continue this way...

We get $14 \cdot 13 \cdot 12 \cdot \dots \cdot 3 \cdot 2 \cdot 1$ which is a little over 87 billion possible routes for ONLY 15 points!

In general, if we have N points, then there are $(N - 1)!$ possible routes.

BF Approach Coding Notes

Writing an algorithm to get all the possible routes is a great exercise, but it can be tricky to a beginning coder. So, I will allow you to use the MATLAB command [perms](#) to do this.

perms command

Let v be a vector of N values. The command

$P = \text{perms}(v)$ returns an $(N \times N!)$ matrix containing all permutations (i.e. orderings) of the values inside the vector, v . Each of the $N!$ rows of the output matrix, P , contains a different ordering of the values in v .

Example: We can get the same list of possible routes for the 4 point example above by typing the command:

```
possible_routes = perms([2 3 4])
```

```
possible_routes = 6x3
    4     3     2
    4     2     3
    3     4     2
    3     2     4
    2     4     3
```

If you think it is help in your code, you can also concetenate a column of ones to the left of this matrix with the command

```
possible_routes = [ones(6,1) possible_routes]
```

```
possible_routes = 6x4
    1     4     3     2
    1     4     2     3
    1     3     4     2
    1     3     2     4
    1     2     4     3
    1     2     3     4
```

Nearest Neighbor (NN) Approach (also known as the greedy approach)

The Nearest Neighbor (NN) approach to the TSP chooses its route based on moving to the closest point to the current point.

PROS: Easy to code and can be applied to many different points.

CONS: Does not guarantee the shortest route.

Again, consider the 4 points P_1, P_2, P_3, P_4 . Also, let $D_{ij} = \text{distance}(P_i, P_j)$.

General Idea for 4 points:

- Initialize the route order as $[1, 0, 0, 0]$ (we always start at P_1)
- Compute the distances between P_1 and P_2, P_3, P_4 . That is, compute $[D_{12}, D_{13}, D_{14}]$
- Find the smallest of these distances. Say it is D_{14} .
- Update the route order to $[1, 4, 0, 0]$.
- Compute the distances between P_4 and P_2, P_3 . That is, compute $[D_{42}, D_{43}]$
- Find the smallest of these distances. Say it is D_{42} .
- Update the route order to $[1, 4, 2, 0]$.
- Compute the distances between P_2 and P_3 . That is, compute $[D_{23}]$
- The smallest of these distances is of course D_{23} .
- Update the route order to $[1, 4, 2, 3]$.

NN Approach Coding Notes

Clearly, your TSP_get_global_dist_vector program will be useful.

You may use MATLAB's min command.

Since you never go back to an already visited location, you may want to introduce a variable that keeps track where you have been so that you don't include those locations in future calculations.

